

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    printf(stderr, "ERRORE: serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    printf(stderr, "ERRORE: impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



# Inter-Process Communication

## FIFOs

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

## License Information

This work is licensed under the license



### Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

# FIFOs

- ❖ FIFOs are an extension of traditional pipes and are sometimes called **named pipes**
  - They allow a communication among **unrelated** processes
  - They can **last** as long as the system does
    - They can be deleted if no longer used
- ❖ A FIFO is a type of file
  - Creating a FIFO is similar to creating a file
    - A FIFO corresponds to a file in the local storage
      - They have a **pathname** in the filesystem
    - Once a FIFO has been created, processes can open it and perform R/W operations on it

## Logic flow

Similar to the one  
for pipes

### ❖ Logic flow to use a FIFO

This step differentiates  
FIFOs from pipes

#### ➤ Create the FIFO

- A FIFO special file is entered into the filesystem by calling **mkfifo**
- **Subsequent** calls to **mkfifo** for the “same” FIFO **have no effect**

#### ➤ Open the FIFO

- Once a FIFO special file has been created, any process can open it, using the **open** system call

#### ➤ Use the FIFO

- Once a FIFO has been opened, we can use **read** and **write**, as for ordinary files

## Function **mkfifo**

```
#include <sys/stat.h>
```

```
int mkfifo (const char *path, mode_t mode);
```

```
int mkfifoat (int fd, const char *path, mode_t mode);
```

❖ Function **mkfifo** creates a FIFO

➤ The parameters **path** and **mode** are similar to the corresponding ones specified for function **open**

- Please refer to **open** for any further explanation on the **mode** parameter
  - Use constant S\_I[RWX]USR or an octal representation to specify user and group ownership

## Function **mkfifo**

```
#include <sys/stat.h>
```

```
int mkfifo (const char *path, mode_t mode);
```

```
int mkfifoat (int fd, const char *path, mode_t mode);
```

Return value

The value 0, on success

The value -1, on error

❖ Function **mkfifoat** is a minor variation of **mkfifo**

- Parameter **fd** indicates a file whose directory path is use to open the FIFO file
- The **path** is relative to the directory specified by **fd**

Please, refer to the system call  
open, read, write, close

## FIFO manipulation

- ❖ Once the FIFO is in the system, we can use normal file IO functions to operate on it
  - The FIFO must be open at **both** ends before performing any input or output operation
  - As with a pipe, if we write to a FIFO that no process has opened for reading, the signal SIGPIPE is generated
  - When the last writer closes the FIFO, an end of file is generated for the readers
  - If we have multiple writers on the same FIFO, we have to worry about atomic writes to avoid interleaving the outputs from multiple processes

## FIFO manipulation

- ❖ The **open** operation on a FIFO can be blocking or non-blocking

```
fd = open (myfifo, ... | O_NONBLOCK);
```

- Without the **O\_NONBLOCK** flag
  - On **open** in read-only (write-only) mode is **blocking** until some other process open the FIFO in write-only (read-only)
- With the **O\_NONBLOCK** flag
  - An open in read-only mode return immediately
  - An open in write-only mode returns -1 (and errno set to ENXIO)

## FIFO manipulation

❖ FIFO may be eventually removed

```
sprintf (str, "rm -rf %s", myfifo);  
system (str);
```

myfifo = "/tmp/myfifo"

To remove "/tmp/myfifo", i.e.,  
prw-rw-r-- 1 quer quer 0 apr 5 16:14 myfifo

# Example

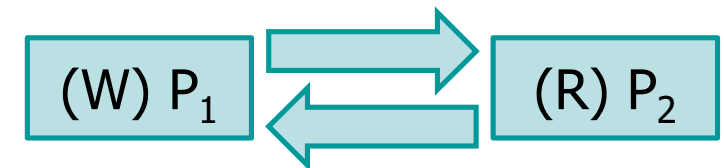
## ❖ Create and use a FIFO between

- A writer (producer)
- A reader (consumer)

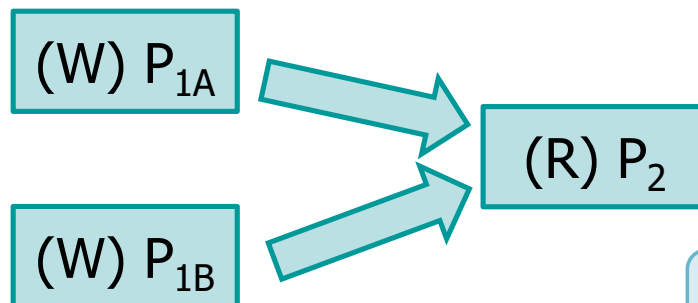


## ➤ Variations

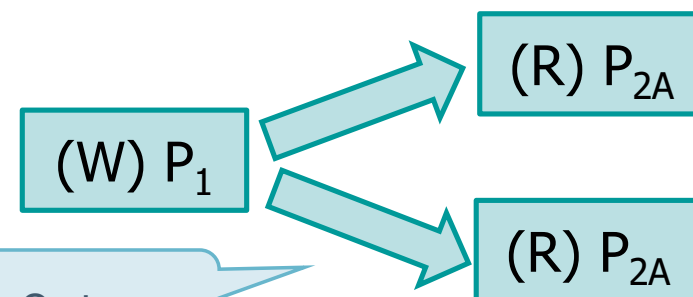
- Use same FIFO in both directions
- Use more FIFOs



Intercept  
more I  
streams



Unlike pipes, FIFO have a  
name can be used for  
non-linear connections



Duplicate O stream

# Example

Run this process on  
a shell windows

1 Reader + 1 Writer  
(client server communication)

```
int main() {  
    int fd; char str[80];  
    char *myfifo = "/tmp/myfifo";  
    mkfifo (myfifo, 0666);  
    fd = open (myfifo, O_WRONLY);  
    while (1) {  
        printf ("Send to reader: ");  
        fgets (str, 80, stdin);  
        write (fd, str, strlen (str)+1);  
        if (strncmp (str, "end", 3)==0) {  
            break;  
        }  
    }  
    close (fd);  
    return 0;  
}
```

(W) P<sub>1</sub>

(R) P<sub>2</sub>

The Writer

Read from stdin

Write to FIFO

Stop the process  
when "end" is  
introduced

FIFOs still transmit  
unstructured data

# Example

Run this process on  
**another** shell windows

```
int main() {  
    int fd;  
    char str[80];  
    char *myfifo = "/tmp/myfifo";  
    mkfifo (myfifo, 0666);  
    fd1 = open (myfifo, O_RDONLY);  
    while (1) {  
        read (fd, str, 80);  
        printf ("Received from writer: %s", str);  
        if (strncmp (str, "end", 3)==0) {  
            break;  
        }  
    }  
    close(fd);  
    return 0;  
}
```

The two processes  
share the FIFO  
name not a parent

(W) P<sub>1</sub>

(R) P<sub>2</sub>

The Reader

Read from the FIFO

Stop the process  
when "end" is  
received